LayerTree: A Shared Bitcoin UTXO with an $O(1)$ Unilateral Exit

Olaoluwa Osuntokun

Lightning Labs
laolu@lightning.engineering

2026-10-10
Draft (commit dce2c57)

Abstract. LayerTree lets any number of users share one Bitcoin output, and any one of them leaves alone in one transaction: the unilateral exit is $O(1)$ in transactions, whatever the number of members, and $O(D)$ in bytes for a tree of depth D (about 1,020 virtual bytes at depth 16). The output commits to an off-chain Merkle-sum tree of balances, and tapscript covenants enforce every change to it, with no pre-signed interior and no expiry of the shared output; an expired lease is evicted, and a cohort joins, settles or is swept, in one transaction too. We describe each operation in the order a member meets it, who authorises it and what it costs, and the voucher system: a signature over a source leaf and a new leaf that anyone can redeem once, which an issuer re-signs for a running total, so repeated payments to one recipient, including payments received over Lightning, cost nothing on-chain and never expire. The design compiles to four tiers of proposed opcodes, the smallest needing `OP_CAT` and one point-tweak opcode; channels live in leaves; the tree rules are mechanised in Lean 4, and a simulator, a byte-exact Go implementation and 3,019 executable checks accompany it.

Keywords: Bitcoin · covenants · Merkle-sum trees · vouchers · payment channels · formal verification

1 Introduction

LayerTree lets N users share one Bitcoin unspent transaction output (UTXO) and lets any one of them leave alone in a single transaction: the unilateral exit is $O(1)$ in transactions for any N, and $O(D)$ in bytes for a tree of depth D. Payment channels [30] share an output between two parties; most approaches for *groups* pay for the sharing when it is dissolved. In channel factories [6], Ark [17,31], timeout trees [18,44] and covenant trees built on `OP_CHECKTEMPLATEVERIFY` (CTV) [32] or `OP_CHECKCONTRACTVERIFY` [15,14], the shared output is the root of a tree of *transactions*. A member who leaves unilaterally publishes the path from the root to its leaf, $O(\log N)$ transactions each paying a fee; the interior must exist in advance as pre-signed transactions (every member signs, online) or covenant templates; and when the tree expires, reclaiming the funds of one unresponsive member

means publishing everything. Payment-pool opcode proposals (`OP_TAPLEAF_UPDATE_VERIFY` [38], CoinPool's `OP_MERKLESUB` [22], `OP_EVICT` [43]) target one-transaction withdrawal instead, with purpose-built opcodes.

LayerTree takes a different route. The tree stays off-chain and is a tree of *data*: a binary Merkle-sum tree of balances whose root one output commits to. A member exits by presenting its leaf and sibling path to the output itself; the script recomputes the root from the proof, checks it against the commitment, recomputes it with the leaf removed, and requires the transaction to recreate the output under the new root with the leaf's amount paid out. The member needs nobody else's signature, and a deeper tree adds bytes to the exit, not transactions. The same covenant lets the operator evict members whose lease has expired, graft a pre-built subtree of new members in one step, and sweep an expired cohort cheaply.

This paper explains the design for those who would build on it: the tree (§3), every operation in the order a member meets it (§4), vouchers, which pay a member or receive from Lightning with no one's participation at redemption (§5), and who can do what (§6). The operations compile to four tiers (§7), from restored string opcodes [35,36], introspection [34], stack signature checks [4] and elliptic-curve arithmetic [25,33] down to `OP_CAT` and one point-tweak opcode, plus one on `OP_CHECKCONTRACTVERIFY` [15]. We propose no opcode, but three tiers assume opcodes in a tapleaf version their proposals do not define. The elliptic-curve draft [25] is by the author. The ideal functionality, its UC proof and the Lean development are in the full version (§8). The specification, code and proofs are public [26]; this text describes revision dce2c57.

2 Background

Bitcoin. A pay-to-taproot output [40] commits to an internal key P and the Merkle root m of a *taptree* of scripts through $Q = P + H_{\mathsf{TapTweak}}(P \,\|\, m) \cdot G$, and is spent either by a Schnorr signature [39] under Q or by revealing one script with its Merkle path (the *control block*) and satisfying it. Tapscript [41] has no loops, so every "loop" below is unrolled. BIP-441 [36] defines tapleaf version `0xC2`, restoring the string opcodes disabled in 2010 (`OP_CAT`, `OP_SUBSTR`, `OP_LEFT`, `OP_RIGHT`, bit operations) with unsigned numbers of any length, metered by BIP-440 [35] ("varops"); BIP-347 [13] proposes `OP_CAT` alone in the current version, `0xC0`. `OP_TX` [34] pushes fields of the spending transaction, optionally collated into one hash preimage; `OP_CHECKSIGFROMSTACK` (CSFS) [4] verifies a signature over a stack element; `OP_TWEAKADD` [33] computes $x(P + tG)$, and the EC-ops draft [25] adds point arithmetic. All are proposals. Sizes are in virtual bytes (vB), a quarter of a transaction's weight in weight units (WU), where a witness byte weighs one unit and any other byte four. A Merkle-sum tree [20,24] hashes each node's children with their sums, so an inclusion proof proves a leaf's contribution to the root sum.

3 How LayerTree Works

A *tree* has one *operator* and any number of *members*. Each member owns a *leaf*: a balance and three keys (an owner key that authorises the member's actions, an exit key that an exit pays, and an evictor key, normally the operator's, that may remove the leaf once its lease height has passed). The operator holds no key over the funds, only an *operator key* that admits new members and co-signs cooperative changes. The balances live in a Merkle-sum tree kept off-chain; one taproot output holds their total and commits to the root, and each operation is a tapscript leaf of that output, which we call its *predicate*.

3.1 The Tree

We write $\mathsf{le}_k(x)$ for the k-byte little-endian encoding of x. A leaf is a fixed 116-byte record

$$\mathsf{le}_{64}(\mathsf{amount}) \,\|\, \mathsf{exit_key} \,\|\, \mathsf{owner_key} \,\|\, \mathsf{evictor_key} \,\|\, \mathsf{le}_{32}(\mathsf{evict_after}) \,\|\, \mathsf{le}_{32}(\mathsf{nonce}) \,\|\, \mathsf{le}_{32}(\mathsf{seq}),$$

with keys as 32-byte x-only points, $\mathsf{evict_after}$ a block height, nonce a per-leaf identifier and seq a counter advanced by vouchers (§5). A *node* is a pair (h, s) of a 32-byte hash and an unsigned sum. A position holding leaf ℓ has node $(H_{\mathsf{leaf}}(\ell), \ell.\mathsf{amount})$; an empty position has node $E_0 = (0^{32}, 0)$. The parent of (h_L, s_L) and (h_R, s_R) is

$$\big(H_{\mathsf{node}}(h_L \,\|\, \mathsf{enc}_{64}(s_L) \,\|\, h_R \,\|\, \mathsf{enc}_{64}(s_R)),\ s_L + s_R\big), \qquad \mathsf{enc}_{64}(s) = \mathsf{le}_{64}(s) \,\|\, \mathtt{0x01}.$$

Every hash is a BIP-340 tagged hash with its own tag; $\mathsf{enc}_{64}(s)$ is the minimal BIP-441 encoding of $s + 2^{64}$, produced from a computed sum with one addition. E_h is the node of the empty tree of height h.

The *sibling path* π of a position is the list of D pairs $(\mathsf{dir}_k, (h_k, s_k))$, one per level, giving the side the position lies on and the sibling node not taken. $\mathsf{fold}(n, \pi)$ combines a node with a path from the bottom up, hashing with each sibling in the order the direction gives and adding the sums. The honest path folds a leaf's node to the root, and folding a *different* node up the *same* path yields the root with that position replaced (§8).

Profiles. In the *dense* profile the operator assigns positions and a path carries direction bits. In the *sparse* profile a leaf's position is derived by hashing its owner key and nonce, as in the Merkle-sum sparse Merkle tree of Taproot Assets [24], truncated to $D = 32$ or 64, at 1.7 to 2.8 times the dense exit cost. Dense is the default and is what we measure.

3.2 The Tree Output

Let r be the root hash and S the root sum. The tree output is a taproot output with internal key $P = \mathsf{lift_x}\big(x(\mathsf{NUMS} + H_{\mathsf{state}}(r) \cdot G)\big)$, where NUMS is the BIP-341 nothing-up-my-sleeve point and $\mathsf{lift_x}$ the BIP-340 even-y normalisation, so

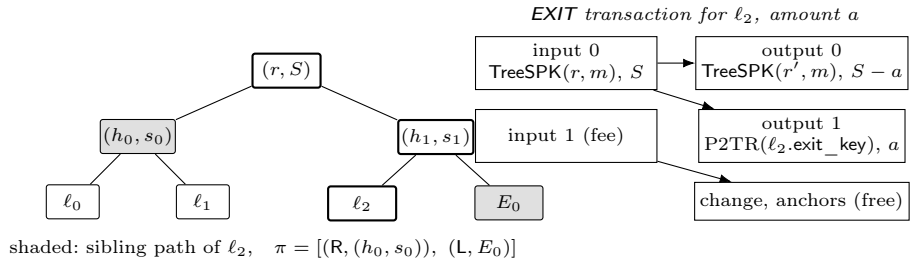


Fig. 1. A depth-2 tree and the exit of ℓ_2. The predicate checks $\mathsf{fold}((H_{\mathsf{leaf}}(\ell_2), a), \pi) = (r, S)$, computes $r' = \mathsf{fold}(E_0, \pi)$, verifies a signature by $\ell_2.\mathsf{owner_key}$, and constrains outputs 0 and 1. Everything else in the transaction is free.

the key path is unspendable and every spend goes through a script. Its taptree has one leaf per predicate; its Merkle root m identifies the rule set and is fixed for the tree's life. A *tree output* for state (r, S) has scriptPubKey $\mathsf{TreeSPK}(r, m)$ and amount S. Every predicate recomputes $\mathsf{TreeSPK}(r, m)$ from its witness and checks it against the spent input, binding the root the proofs are checked against and the script set the successor carries. FULL, MID and MIN predicates read input 0, so two such trees cannot be spent in one transaction; a CCV tree's input may sit at any index.

4 Operations

Every operation is one spend of the tree output through one tapleaf. The tree output is input 0, and output 0 recreates it under the new root and total. Payouts are outputs $1, \ldots, K$, each paying exactly a removed leaf's amount to its exit key; every other input and output is free, for fees, change and anchors. The witness proves each changed position with a *step*: the old node folded up its sibling path must give the committed (r, S), and the new node folded up the same path gives (r', S'); steps chain. Signatures cover all outputs. If the last leaf leaves, output 0 is unconstrained. Costs are whole transactions at $D = 16$ in FULL, on the model of §10.

4.1 Joining

JOIN$_h$: grafting a cohort. The operator inserts a subtree of up to 2^h new leaves, built off-chain, into an empty aligned slot of height h; $h = 0$ adds one leaf. *Authorised by* the operator key alone; no existing member signs or is affected. *Proves* that the slot held the empty node E_h, and adds the new subtree's root and sum. *On chain* the tree output grows by the subtree's sum, which other inputs must supply. The covenant sees a hash and a sum, not the leaves, and walks only the $D - h$ levels above the slot, so a larger cohort is a smaller transaction: 937 vB

for one leaf, 872 vB for four (218 each), 679 vB for 256 (2.7 each). A taptree carries one JOIN_h leaf per height it supports.

Because the covenant cannot look inside the graft, each joiner checks it. A member funding its own leaf verifies, before signing its input over all outputs, that the slot folds as empty to the current root, that its leaf folds to the grafted node with every sum recomputed, and that output 0 commits to the new root; its input can then only be spent into a root containing its leaf. A self-funded cohort is thus a signing round, like an Ark round or a coinjoin, and each member adds its funding input (58 vB). In an operator-funded graft the member pays off-chain, after checking its path against the confirmed root, or through a hash-locked voucher (§5.4).

JOINK_K: batched single joins. K single-leaf joins in sequence under one operator signature. It is the batching primitive of the sparse profile, where a leaf's position is derived from its key and cannot be grafted as part of a subtree, and costs 1,494 vB for two in the dense profile.

4.2 Moving Value

Most payments never touch the tree. Members with channel leaves pay over Lightning through the operator (§9), and vouchers (§5) move value with a signature that is redeemed only if the issuer stops cooperating. The tree changes when a balance's terms change or a carve is redeemed.

UPDATE and UPDATE^x: changing a leaf. Replaces one leaf with another at the same position: a splice in or out, a lease renewal, a key rotation or a transfer of ownership. *Authorised by* the leaf's owner and the operator together; UPDATE^x, a variant chosen at creation, adds the leaf's exit key as a third signer. *Proves* the old leaf against the root and writes the new one. *On chain* the tree output changes by the difference: an increase is funded by other inputs, a decrease is free for the transaction to spend, and both signers see where it goes. 1,030 vB, or 1,048 vB with the third signature. UPDATE^x exists for leaves whose exit key is shared with or owned by someone else (a channel's counterparty, a credit leaf's recipient, §5.3): without it the owner and the operator could splice such a leaf down together.

SPLIT: paying a member directly. Carves a new leaf out of the payer's leaf. *Authorised by* the payer's owner key, signing this transaction. *Proves* two positions in sequence: the source leaf, rewritten with its amount reduced by the payment and its seq raised by one (or emptied if drained), then an empty position under the intermediate root, which receives the new leaf. *On chain* the total is unchanged and no payout is made. The new leaf keeps the source's evictor key and a lease no later than the source's, and neither it nor a non-zero residual may be below DUST_MIN (both rules are shared with CLAIM, §5.1). 1,586 vB. SPLIT is the right shape when the payer is the one broadcasting and paying the fee; it leaves no outstanding promise behind, and it is the one member-to-member transfer

the MIN tier supports. Its deferred counterpart, a voucher redeemed by CLAIM, is the subject of §5.

4.3 Settling a Cohort

REWRITE$_h$: everything at once. Replaces the contents of one aligned subtree of height h in a single spend: any position may go from a leaf to empty (an exit, paid out), from empty to a leaf (a join, funded) or from one leaf to another (a payment, renewal, resize or key rotation). *Authorised by* the operator and by the owner of every leaf that changes; a leaf that only gains amount needs no signature unless its seq is SEQ_FINAL (a channel leaf, §9, whose capacity its signed states fix), so a payment within a cohort needs only the payer's, and a leaf nobody signs for passes through unchanged. *Proves* the old subtree, every leaf of it published, against the root, and computes the new one. *On chain* each emptied leaf is paid exactly its amount at its exit key, so the operator cannot use the rewrite to remove a leaf without its owner's signature and its payout. At $h = 6$ a mass exit costs 162 vB per leaf, about what a sweep costs the operator, and rewriting every balance 148 vB per leaf against 1,030 vB for an UPDATE each; a position whose leaf does not change is published once, behind a one-byte marker, for about 92 vB. REWRITE^x_h adds the exit key's signature (16 vB per signed position) and replaces REWRITE$_h$ wherever UPDATE^x or CLAIM^x is present. FULL and MID, where the stack limit holds a full cohort to $h = 5$.

4.4 Leaving

EXIT$_K$: unilateral exit. Removes one leaf, or K leaves in one transaction, and pays each out (Figure 1). *Authorised by* the leaf's owner key alone; no operator, no other member, no timelock. *Proves* the leaf and its path against the root and folds the empty node up the same path for the new root. *On chain* the tree output shrinks by the leaf's amount and output 1 pays its exit key. In FULL the payout may be less than the balance, the difference paying the fee, which saves the fee input (99 vB); the exact-payout variant EXIT$^=_K$, chosen at creation and visible in the tree's taptree root, requires the full amount and is what MID, MIN, CCV and channel leaves use. 1,020 vB for one exit, and 665 vB more for a second in the same transaction (EXIT$_2$ is 1,684 vB). The owner's signature commits to the tree outpoint, which every operation changes, so a plain exit cannot be pre-signed: the owner must be online to rebuild it after a competing spend. The tower variant EXIT^t_K (FULL, MID and CCV, exact payout) replaces that signature by one over the leaf and the tree's identifier, checked from the stack with `OP_CHECKSIGFROMSTACK`; whoever holds it builds the exit with its own fee input and rebuilds it after a competing spend, so an owner signs once in advance and a tower does the rest. The authorisation is a bearer right until the leaf changes, so an owner who stays voids it by changing the leaf. 107 bytes of script over the exact exit. Owners first ask for a cooperative exit, which the operator can batch, and a held authorisation makes that one message.

4.5 Being Evicted

Eviction by EXIT. A lease is how the operator recovers its capital and slots from a member who has gone away, and eviction is the only way it can remove a leaf without the owner. The exit script, with a flag, removes a leaf whose lease has passed. *Authorised by* the leaf's evictor key, with a lock time at or after the lease height. *On chain* the payout is exactly the leaf's amount, to the leaf's own exit key, so an eviction returns the member's money rather than taking it. Same cost as an exit; the evictor pays the fee.

EVICT_SUBTREE$_h$*: the cohort sweep.* Evicts every remaining leaf of one aligned subtree in one spend. *Authorised by* the operator key, which must be every swept leaf's evictor, with a lock time past every lease. *Proves* the subtree from all its leaves (empty positions contribute E_0), each internal node hashed once, against the root. *On chain* one exact payout per leaf, and the subtree becomes empty. 203 vB per leaf for 13 leaves at $h = 4$ and 158 vB per leaf for 51 at $h = 6$, against 1,020 vB each alone. One leaf with a later lease holds up its cohort's sweep until that lease passes, and one with another evictor blocks it, so operators admit only leaves that name the operator as evictor into a cohort, align leases per cohort, and renew a member by moving it to a cohort with the new lease (an UPDATE followed by a SPLIT of the whole balance).

4.6 Scaling Out

DETACH$_h$*: a cohort's own output.* Moves one aligned subtree into a tree output of its own. *Authorised by* the operator key. *Proves* the subtree's root and sum against the parent root. *On chain* output 0 is the parent minus the cohort and output 1 the cohort, under its own root and the parent's rules compiled at depth h; the detached tree inherits the parent's tree identifier, so vouchers and channel states against its leaves stay valid. No member needs new data: its proof in the detached tree is its old path cut to the lower h levels. 780 vB at $h = 8$, after which every exit from the cohort costs 762 vB instead of 1,020; the detach pays for itself at the fourth exit. Its larger effect is throughput, since each output is spent independently (§6).

5 Vouchers

A voucher lets a member pay someone, or the operator pay a member who receives from Lightning, with a signature alone. Nothing reaches the chain when it is issued, and its holder can redeem it later without the issuer or the operator. Vouchers are the LayerTree counterpart of Ark's out-of-round payments, with one difference: a redeemed voucher is a leaf of the tree, which its holder can exit alone, not an output that expires with a round.

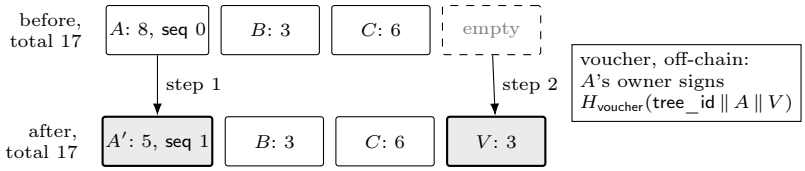


Fig. 2. Redeeming a voucher for 3 from A to V. CLAIM shrinks the source and advances its seq, then fills an empty position with the claim leaf, with no signature but the voucher's. The tree now holds A', not A, so the signed message never matches again.

5.1 The Voucher and CLAIM

A voucher is a BIP-340 signature by the owner key of a *source* leaf over $H_{\mathsf{voucher}}(\mathsf{tree_id} \,\|\, \ell_{\mathrm{src}} \,\|\, \ell_{\mathrm{claim}})$, where ℓ_{src} and ℓ_{claim} are full 116-byte leaves and tree_id is a 32-byte constant fixed when the tree is created. ℓ_{claim} is the leaf being paid: an amount and the recipient's keys. Anyone who holds the voucher can redeem it with CLAIM, which checks the signature with `OP_CHECKSIGFROMSTACK` and needs no other. Like SPLIT, it is two steps (Figure 2). The first proves ℓ_{src} against the committed root and rewrites it with its amount reduced by the claim's and its seq raised by one, or empties the position if the voucher draws the whole balance. The second proves a position empty under the intermediate root and places ℓ_{claim} there. The total is unchanged.

Two rules bind every carve, by voucher or by SPLIT. The claim leaf keeps the source's evictor key and a lease no later than the source's, so moving a balance cannot escape eviction, nor plant a leaf in a cohort that the operator could never sweep. And neither the claim leaf nor a non-zero residual may be below DUST_MIN, by default 330 sat, the P2TR dust threshold: a smaller leaf could leave only through a payout that default policy does not relay, so a one-satoshi carve could block its cohort's sweeps and grafts for the life of the tree.

Single use. Replay is impossible because redemption changes the leaf the voucher names, and seq keeps it changed even if the balance later returns to its old value. Two rules that no script enforces carry the rest: a leaf's seq only ever advances (an UPDATE winding it back would revive old vouchers), and no two leaves share an owner key and nonce, since a voucher binds its source by content, not position. Binding tree_id rather than the taptree root m keeps a voucher out of other trees built with the same parameters, and since a detached cohort inherits tree_id, vouchers survive a DETACH_h.

5.2 Cumulative Vouchers

Every voucher against a source draws on the same leaf, and the first redemption voids the rest. An issuer that pays one recipient repeatedly therefore signs a *cumulative* voucher for the running total against the same claim leaf, and re-signs it for a larger amount at each payment: 2, then 5, then 9. Each payment

is one signature off-chain, with no on-chain cost, no minimum and no expiry. The recipient keeps only the latest voucher and, if the issuer stops cooperating, redeems that one alone; it never prefers a smaller one, so it needs no window in which to contest an old state.

Settled credit is retired by advancing seq. Once the issuer pays the recipient another way, for instance with a leaf of the recipient's own in the next cohort graft (under 3 vB per member), it advances the source's seq, which voids every outstanding voucher against it: one UPDATE per source leaf (1,030 vB), or one $\mathsf{REWRITE}_h$ for a cohort of them (about 148 vB per leaf at $h = 6$). Until then the issuer tracks every voucher it has signed as a liability. Plain CLAIM leaves a hole: anyone holding an old voucher can redeem it, and the issuer holds them all, so redeeming the voucher for 2 voids the one for 9. The credit leaf below closes it.

5.3 What the Holder Trusts

The covenant prevents over-*redemption*: the first redeemer is paid by consensus. It cannot prevent over-*issuance*, or voiding: the issuer can advance seq first, by CLAIM, SPLIT or UPDATE. So the holder of an ordinary voucher extends credit to the issuer until it redeems or is paid. Eviction voids vouchers too: once the source's lease passes, its evictor can pay the whole balance to the source's exit key, normally the issuer's. A holder therefore treats a voucher whose source lease can pass before a redemption would confirm (within $\Delta(q)$, §6) as unsecured. A partial-balance voucher also needs an empty position for its claim leaf, which the operator controls, so a holder prefers a full-balance voucher.

The dedicated credit leaf. A dedicated credit leaf is a source leaf that the issuer owns and whose exit key is one recipient's, in a tree whose taptree carries CLAIM^x, UPDATE^x and the exact-payout exit, carries $\mathsf{REWRITE}^x_h$ in place of $\mathsf{REWRITE}_h$ (or no rewrite at all), and omits SPLIT. CLAIM^x is CLAIM with one further signature over the transaction, by the source leaf's exit key: 7 bytes of script and 64 of witness in FULL, the only tier that builds it. The recipient's own redemptions carry that signature at no cost, and the issuer cannot carve the leaf at all, not even with a voucher it signs to itself. Over-issuance then has no second victim, since every voucher pays the same recipient, who can watch the leaf's amount. Voiding needs the leaf to change, and every transition that could change it needs the exit key: CLAIM^x, UPDATE^x and $\mathsf{REWRITE}^x_h$ by rule, while the exact exit and an eviction pay the whole balance to that key and so void nothing the recipient is owed. The balance can only grow or pay its recipient, whose remaining exposure is the issuer's liveness: a provider that stops issuing.

Three details keep it that way. The recipient checks from the taptree root m that no plain $\mathsf{REWRITE}_h$ is present, since it needs only the owner's and the operator's signatures, both the issuer's. The issuer funds the leaf only with credit it has issued or is about to, since any excess is the recipient's on every path the issuer can take alone, and tops it up by an amount-only increase under $\mathsf{REWRITE}^x_h$, which needs only the operator's signature. And the leaf has a finite lease with the operator as evictor, not a NUMS evictor, which every carve would

inherit, blocking the sweep of whatever cohort the claim leaf lands in. The credit is one-way: if the recipient could also spend from it, an older, larger voucher would be a stale state needing a window to supersede, as in a bidirectional channel. To spend, the recipient redeems into a channel leaf (§9).

5.4 Hash-Locked Vouchers and Swaps

CLAIM^h adds a payment hash to the signed message, under its own tag so that the lock cannot be stripped, and requires the 32-byte preimage: the script hashes it into the message, so a wrong preimage fails as a bad signature. The same hash can sit in a channel-leaf HTLC or a Lightning invoice, but the pairing is an ordered exchange, not an atomic swap: the issuer can void the voucher until the redemption confirms, and a voucher has no deadline, since script can forbid a spend before a height but not after one. Five rules (spec 9.6.2) order it. (1) The party the HTLC pays draws the preimage. (2) The voucher resolves second; no HTLC waits on a preimage that only a redemption would reveal. (3) The issuer leaves the source's seq alone from the moment an HTLC on the hash is offered to it until the voucher is redeemed or the HTLC fails. (4) No script enforces rule 3, so the holder redeems as soon as it learns the preimage and bears the issuer risk until that confirms. (5) The holder first checks that the source's lease outlasts the HTLC's expiry plus $\Delta(q)$. Usually the HTLC settles off-chain, so such a swap costs at most one CLAIM^h; it is also how a newcomer buys a leaf with Lightning funds.

5.5 Receiving over Lightning

The flow. A user with no on-chain funds receives a Lightning payment into a credit leaf that the operator owns and that names the user's key as exit key. The payer's HTLC reaches the operator, which obtains the preimage and settles that inbound HTLC *first*; only then does it sign the user a voucher for the new running total, a plain cumulative voucher against the credit leaf, redeemable under CLAIM^x. Nothing reaches the chain, there is no minimum per payment beyond DUST_MIN at redemption, nothing expires, and the recipient need not be online when it is paid. The order matters. A hash-locked voucher issued before the HTLC settles, with the preimage held by the recipient, breaks the rule that the party the HTLC pays draws the preimage (§5.4), and on a credit leaf it cannot be repaired: if the HTLC then fails, retiring the voucher means advancing the credit leaf's seq, which needs the recipient's exit key, so the voucher stays redeemable for a payment that never arrived. Settling first leaves the recipient exposed for one payment, the one the operator has settled and not yet vouched for.

Settlement. Normally the voucher is never redeemed. The operator places a leaf of the recipient's own in its next cohort graft, at under 3 vB per member in a 256-member JOIN_8, and retires the voucher by advancing the credit leaf's seq and lowering its amount by what it has paid out: one UPDATE^x per credit

Table 1. Who can do what (spec 10.1). "Carve" is SPLIT or a redeemed voucher.

party	can	cannot
Operator	refuse to co-sign; graft leaves, including ones it funds; evict a leaf after its lease, paying it in full, with everything carved from it; issue vouchers on its own leaves; detach a cohort; fill a cohort's empty positions, stalling partial-balance vouchers	move or reduce another leaf; delay an exit beyond [C] (it contends by fee, like anyone); freeze value it did not graft and fund; change the taptree, a lease, or a detached cohort's rules, keys or balances
Owner	exit at any time; co-sign updates; issue vouchers against its leaf; carve its leaf	affect any other leaf; withdraw more than its amount; carve a leaf with another evictor or a later lease than its own
Anyone	redeem a valid voucher; relay transitions	trigger an owner's exit; create leaves; alter amounts

leaf (1,048 vB), which the recipient co-signs, or one $\mathsf{REWRITE}^x_h$ for a cohort of credit leaves, about 148 vB per changed leaf at $h = 6$ plus 16 vB for each exit-key signature. If the operator stops cooperating, the recipient redeems its latest voucher alone with one CLAIM^x: 1,635 vB at $D = 16$, about 16,350 sat at 10 sat/vB, with no timelock, against 1,311 to 2,511 vB and a relative timelock to unroll an Ark round tree of 256 to 65,536 members (§10).

Costs. Issuing a voucher costs nothing on-chain. A redemption proves two positions, so at $D = 16$ it costs more than an exit: 1,617 vB for CLAIM, 1,627 vB for CLAIM^h and 1,635 vB for CLAIM^x, against 1,586 vB for SPLIT, on the model of §10, which charges a carve no payout output (with one, add 43 vB). Redemption is the fallback; normally vouchers are re-signed off-chain and retired in batches.

6 Trust Model

Custody never rests with the operator. The tree output's key path is unspendable, and every script path needs either a proof against the committed root plus the affected owner's signature, or the operator's signature for operations that only add value or move a cohort intact. Table 1 lists what each party can and cannot do.

What a member does. A member stores its leaf and path and follows every spend of the tree output; each witness carries the path it used, so the member updates its own path from the chain alone and needs nothing else to exit. It verifies its path before funding a join (§4.1) and the source before accepting a voucher (§5.3), and must be online to exit, since its signature commits to the current outpoint.

Liveness. Every operation spends and recreates one output, so an honest exit can be overtaken by conflicting spends (a corrupt member alone can carve its balance back and forth), and its delay depends on policy and fees. We state it as an assumption. *[C] (contention):* at most $\kappa \geq 1$ spends of a tree output confirm

Table 2. The four opcode tiers. "Shape" says whether inputs and outputs the predicate does not constrain are free or fixed by the script. Sizes are bytes of script / witness data for EXIT_1 in the dense profile at $D = 16$; the last row is the transaction size on the model of §10. MID's sweep fits at most 54 occupied positions at $h = 6$ (stack limit). For scale, a hypothetical native Merkle-sum opcode, measured in the simulator and not a tier, would take EXIT_1 to 252 bytes of script at any depth and about 388 vB on a six-leaf tree, about 520 vB in a full depth-16 tree (§12).

	FULL	**MID**	**MIN**	**CCV**
Tapleaf version	`0xC2`	`0xC0`	`0xC0`	`0xC2` or `0xC0`
Beyond BIP-342	`OP_CAT`, splice, bit ops, `OP_TX`, CSFS, EC ops	`OP_CAT`, `OP_TX`, CSFS, `OP_TWEAKADD`	`OP_CAT`, `OP_TWEAKADD`	`OP_CCV` (BIP-443), `OP_TX`
Numerics	any length	4-byte, $< 2^{31}$	4-byte, $< 2^{31}$	as FULL or MID
Owner payout	$\leq$ amount	exact	exact	exact
Predicates	all	all	EXIT_1, SPLIT	all
Vouchers / channels / shape	yes / yes / free	yes / yes / free	no / no / fixed	option / yes / free
EXIT_1 script / witness (B)	2,101 / 902	3,367 / 978	3,640 / 1,175	1,716 / 838
EXIT_1 transaction (vB)	1,020	1,355	1,473	908

per block, and while an honest spend is pending at the prevailing feerate at most β conflicting adversarial spends confirm ahead of it. κ is mempool policy (25 under Bitcoin Core's chain limits before version 31, at most 64 under its cluster limit since [3,9], as low as 1 once anyone hangs descendants off a payout); β is the adversary's fee budget against the honest bid. Under [C] an honest spend with q others queued ahead confirms within $\Delta(q) = \Delta_L(\lceil(\beta + q)/\kappa\rceil + 1)$ blocks, Δ_L being the chain's own delay. Cooperatively a tree drains 25 exits per block, so 10,000 leaves closing at once take 400 blocks: a tree is sized by exit throughput, not depth. The operator can evict any leaf once its lease passes, so a member who wants a longer guarantee needs a later lease at entry; no carve extends one.

7 Realisations by Opcode Tier

Each operation of §4 is compiled to tapscript against four opcode sets (Table 2), so that the design's dependence on each candidate opcode is measured rather than asserted; the tree format and the predicates are the same in every tier. FULL, MID and CCV compile every operation, MIN the dense single-leaf exit and SPLIT; CLAIM^x is FULL's alone, $\mathsf{REWRITE}_h$ exists in FULL and MID and DETACH_h in FULL and CCV. Table 3 (Appendix A) lists what each tier assumes of the proposals it uses, mirroring the assumptions [A-1] to [A-6] of the specification. MIN is the tier closest to the proposals as written: it needs only `OP_CAT` (BIP-347) and `OP_TWEAKADD` (BIP-449), both defined for the current tapleaf version. MID and CCV in `0xC0` need `OP_TX` in a version its draft does not define; CCV in `0xC2` needs BIP-443 there; FULL needs CSFS and EC ops in `0xC2` and two drafts whose opcode numbers collide.

What each tier gives up. In FULL the leaf is one stack element and sums use arbitrary-length arithmetic. MID's 4-byte numbers cap a tree at 2^{31} satoshis

(21.47 BTC) and force exact payouts. MIN has no `OP_TX`: the script builds the BIP-341 signature message from the values it has verified and checks it with a signature under the generator G (Poelstra's construction [28]), which passes only if the message equals the real transaction's (assumption [MIN]); vouchers and channels, which need a signature over stack data, are out of its reach, but SPLIT is not (6,651 bytes of script). CCV replaces every taproot-key recomputation with an `OP_CHECKCONTRACTVERIFY` call and is the smallest tier.

8 What Is Proved

The rules of every operation are mechanised in Lean 4 [21] (about 3,400 lines, no Mathlib), with an axiom audit of 60 headline theorems, none resting on an unproved step. They show that a root binds its tree, so a leaf whose path verifies is the leaf at that position; that every sequence of steps conserves the total, with no hash assumption; that every known leaf can be paid out by its honest witness; that a graft without a real preimage freezes only the value it brought; that a voucher cannot be replayed and a carve keeps the evictor, lease and dust floor; and that the cohort operations and encodings behave as the scripts assume. Sums are conserved unconditionally, but an exit is guaranteed only to a member who verified its path. Scripts are matched to the rules by executable checks (§10), not a verified compiler.

For each tier, the protocol UC-realises an ideal functionality of the shared output in a global-ledger hybrid model [7,2], with idealised hashes, taproot binding in the random-oracle model and [C], against a static adversary corrupting the operator and any members. The full version gives the functionality, the simulator, the proof and the Lean statements.

9 Channel Leaves

A leaf whose exit key is a 2-of-2 between its owner and a counterparty (normally the operator) hosts a payment channel whose funding output appears only when an exit or eviction pays the leaf out, so a tree of such leaves is a Lightning hub behind one UTXO. That outpoint changes with every tree transition, so states are signed without committing to it (`OP_TX` collates the signed fields and CSFS checks the signature, emulating `SIGHASH_ANYPREVOUT` [11]), with the eltoo ratchet [10,37], zero-fee version-3 transactions and an ephemeral anchor [42]. This needs FULL, MID or CCV.

Deadlines and hub size. Settlement outputs include one per in-flight hashed time-locked contract (HTLC). A forwarding node keeps cltv_expiry_delta blocks between an incoming and an outgoing HTLC's expiry, and a channel leaf needs $\mathsf{cltv_expiry_delta} \geq G_{\text{exit}}+3+\mathsf{csv_delay}+G_{\text{sweep}}$, with csv_delay the settlement's relative delay, G_{sweep} the time to confirm an HTLC sweep and G_{exit} the time to confirm the exit, $\Delta(q)$ under [C]. With $\mathsf{csv_delay} = 144$, $G_{\text{sweep}} = 2$ and $G_{\text{exit}} = 72$ this is about 220 blocks, against 40 to 80 on the public network. At

$\Delta_L = 1$, $G_{\text{exit}} = 72$ needs $\beta + q \leq 71\kappa$; with $\kappa = 25$ and $\beta = 0$, about 1,775 channel leaves may close in one window, and larger hubs shard. The counterparty is normally the operator, which can issue competing spends, so the budget holds only while the member can outbid them, which is what β assumes. MID allows seven HTLCs per state beside two balances (the 520-byte element limit) and 2^{31} satoshis per tree.

Conditions. A channel leaf has exact payouts, so the owner cannot underfund the channel; seq = SEQ_FINAL, so no voucher or SPLIT can carve the capacity the signed states commit to; and lease zero with the counterparty as evictor, so eviction is the counterparty's force-close. The state machine and the budget run end to end on the simulator's chain model with two blocks of margin; a proof that channel leaves compose is open. A force-close costs about 1,681 vB, against about 680 vB for a simple taproot channel [27].

10 Evaluation

Implementation and model. A Python tapscript interpreter for both leaf versions computes the real BIP-341 signature message and runs every compiled script against a reference tree, with positive and negative cases per rule (forged leaf, stale root, unconserved amount, early eviction, voucher replay, and others); a Go implementation on `btcd` and `lnd` generates the same scripts byte for byte and builds, signs and verifies every FULL transition. A vB figure is a whole transaction on a 946 WU skeleton (Appendix B.9).

Exit cost. A dense exit (Figure 3, Table 4) costs 762 vB at $D = 8$ and 1,149 vB at $D = 20$, about 32 vB per level. One tier step costs as much as 10 (MID, +33%) to 14 (MIN, +44%) levels of depth. The sparse profile at $D = 32$ with 2^{16} members costs about 1,780 vB, 1.75 times the dense profile at $D = 16$; its witness grows with $\log_2 N$, not D.

Comparison with transaction trees. The dashed line of Figure 3 is a *model*: an exiter publishes D transactions of about 150 vB and a 111 vB claim, anchors and their fee children omitted, which favours the tree. LayerTree is about 60% smaller at $D = 16$, 162 against 261 vB a member under mass exit. The model prices neither the latency of D sequential confirmations nor the larger opcode set; Appendix B compares further.

11 Related Work

Payment pools. CoinPool [22] is a shared UTXO with per-user balances whose one-transaction withdrawal rests on a new opcode, `OP_MERKLESUB`; `OP_TAPLEAF_UPDATE_VERIFY` [38] recreates a pool output with the leaver's key and leaf removed, and `OP_EVICT` [43] removes unresponsive members. LayerTree reaches one-transaction exit from opcodes proposed as general primitives, holds balances

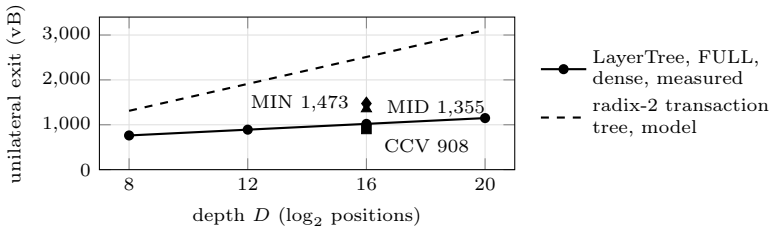


Fig. 3. Unilateral exit cost against tree depth: measured for LayerTree (FULL, and the other tiers at $D = 16$), modelled for an unamortised radix-2 transaction tree.

in a Merkle-sum tree rather than keys in a taptree, and adds lease-based eviction, cohort grafts, channel leaves and a composable statement. *Transaction trees.* Channel factories [6] need all N members to sign every change. Ark [17], deployed today [1], batches users into rounds whose outputs are leaves of a pre-signed MuSig2 tree; covenant Ark replaces the signing with CTV templates [31] but keeps the tree, its unrolled exit and its expiry. Timeout trees [18] and SuperScalar [44] add laddered expiries and Decker-Wattenhofer factories; MATT-style trees [14,15] replace signatures by templates. Table 13 (Appendix B) compares them. *Beyond Bitcoin.* Plasma's forced exits [29] prove a balance against a committed root, enforced by a contract. Proofs of liabilities [16,8] use Merkle-sum trees, where a node that hashes only its children's total admits the inflation that hashing each child's sum avoids. *Components.* The Merkle-sum sparse Merkle tree is Taproot Assets' [24], truncated and verified in script; the data-tweaked key follows BIP-443 [15]; the generator-key signature is Poelstra's [28], and its ECDSA analogue has been used on Bitcoin Cash as `OP_PUSH_TX`. Liquid's tapscript [5] has `OP_CAT`, CSFS, per-field introspection and 64-bit arithmetic, so MID is close to expressible there; Simplicity [23] would express the predicates directly.

12 Discussion and Conclusion

All activity is serialised through one output, so an honest exit's delay is set by policy and fees, which [C] names rather than derives; cluster mempool [9] raises κ for a cooperative tree but shares the cluster with every descendant anyone attaches, and fee pinning is outside [C]. A Liquid prototype of MID [5] is the obvious next experiment.

LayerTree specifies a shared output by what it lets members do (join, leave alone, be evicted after a lease, be swept as a cohort) and realises it with a tree kept off-chain as data: a one-transaction exit of about a kilobyte, no pre-signing, no expiry of the shared output, from proposed opcodes; its price is that all activity funnels through one UTXO. Vouchers pay members off-chain against that output. Future work: a verified compiler, a proof that channel leaves compose, and a P model [12].

References

1. Ark Labs: arkd: Backend services powering Arkade. Repository, https://github.com/arkade-os/arkd (2023)
2. Badertscher, C., Maurer, U., Tschudi, D., Zikas, V.: Bitcoin as a transaction ledger: A composable treatment. In: CRYPTO 2017. LNCS, vol. 10401, pp. 324–356. Springer (2017)
3. Bitcoin Core: Mempool policy defaults: Ancestor and descendant count limits (25) and cluster limit (64). src/policy/policy.h at v31.1, https://github.com/bitcoin/bitcoin/blob/v31.1/src/policy/policy.h (2026)
4. Black, B., Rubin, J.: BIP-348: CHECKSIGFROMSTACK. https://github.com/bitcoin/bips/blob/master/bip-0348.md (2024)
5. Blockstream: Elements tapscript opcodes: Introspection, streaming SHA256, 64-bit arithmetic, ECMULSCALARVERIFY, TWEAKVERIFY. https://github.com/ElementsProject/elements/blob/master/doc/tapscript_opcodes.md (2021)
6. Burchert, C., Decker, C., Wattenhofer, R.: Scalable funding of bitcoin micropayment channel networks. In: Royal Society Open Science. vol. 5 (2018)
7. Canetti, R.: Universally composable security: A new paradigm for cryptographic protocols. In: FOCS 2001. pp. 136–145. IEEE (2001)
8. Chalkias, K., Chatzigiannis, P., Ji, Y.: Broken proofs of solvency in blockchain custodial wallets and exchanges. In: Financial Cryptography and Data Security, FC 2022 International Workshops. LNCS, Springer (2022)
9. Daftuar, S.: An overview of the cluster mempool proposal. Delving Bitcoin, https://delvingbitcoin.org/t/an-overview-of-the-cluster-mempool-proposal/393 (2024)
10. Decker, C., Russell, R., Osuntokun, O.: eltoo: A simple layer2 protocol for bitcoin. https://blockstream.com/eltoo.pdf (2018)
11. Decker, C., Towns, A.: BIP-118: SIGHASH_ANYPREVOUT for taproot scripts. https://github.com/bitcoin/bips/blob/master/bip-0118.mediawiki (2017)
12. Desai, A., Gupta, V., Jackson, E., Qadeer, S., Rajamani, S., Zufferey, D.: P: Safe asynchronous event-driven programming. In: PLDI 2013. pp. 321–332. ACM (2013)
13. Heilman, E., Sabouri, A.: BIP-347: OP_CAT in tapscript. https://github.com/bitcoin/bips/blob/master/bip-0347.mediawiki (2023)
14. Ingala, S.: Merkleize all the things. bitcoin-dev mailing list, https://merkle.fun (2022)
15. Ingala, S.: BIP-443: OP_CHECKCONTRACTVERIFY. https://github.com/bitcoin/bips/blob/master/bip-0443.mediawiki (2025)
16. Ji, Y., Chalkias, K.: Generalized proof of liabilities. In: ACM CCS 2021. ACM (2021)
17. Keceli, B.: Ark: An alternative privacy-preserving second layer solution. bitcoin-dev mailing list, 22 May 2023, https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2023-May/021694.html (2023)
18. Law, J.: Scaling lightning with simple covenants. bitcoin-dev mailing list, https://github.com/JohnLaw2/ln-scaling-covenants (2023)
19. Lightning Network specification: BOLT #3: Bitcoin transaction and script formats. https://github.com/lightning/bolts/blob/master/03-transactions.md (2024)
20. Merkle, R.C.: A digital signature based on a conventional encryption function. In: CRYPTO 1987. LNCS, vol. 293, pp. 369–378. Springer (1988)

21. de Moura, L., Ullrich, S.: The Lean 4 theorem prover and programming language. In: CADE 2021. LNCS, vol. 12699, pp. 625–635. Springer (2021)
22. Naumenko, G., Riard, A.: CoinPool: Efficient off-chain payment pools for Bitcoin. `https://github.com/coinpool-dev/paper/blob/master/coinpool-v0.1.pdf`; `OP_MERKLESUB` draft BIP, `https://github.com/ariard/bips/blob/coinpool-bips/bip-merklesub.mediawiki` (2022)
23. O'Connor, R.: Simplicity: A new language for blockchains. PLAS 2017, `https://blockstream.com/simplicity.pdf` (2017)
24. Osuntokun, O.: Merkle sum sparse Merkle trees. Draft BIP (Taproot Assets), `https://github.com/Roasbeef/bips/blob/bip-tap/bip-tap-ms-smt.mediawiki` (2021)
25. Osuntokun, O.: Elliptic curve operations for Bitcoin script. Draft BIP, `bitcoin/bips` PR #1945 (closed unmerged), `https://github.com/bitcoin/bips/pull/1945` (2025)
26. Osuntokun, O.: LayerTree: Specification, reference implementation and formal model. `https://github.com/roasbeef/layertree` (2026)
27. Osuntokun, O.: Simple taproot channels (extension BOLT, feature bits 80/81). `lightning/bolts` PR #995, merged May 2026, `https://github.com/lightning/bolts/blob/master/bolt-simple-taproot.md` (2026)
28. Poelstra, A.: CAT and Schnorr tricks I and II. Blog posts, 20 Jan and 10 Feb 2021, `https://www.wpsoftware.net/andrew/blog/cat-and-schnorr-tricks-i.html`, `https://www.wpsoftware.net/andrew/blog/cat-and-schnorr-tricks-ii.html` (2021)
29. Poon, J., Buterin, V.: Plasma: Scalable autonomous smart contracts. White paper, `https://plasma.io/plasma.pdf` (2017)
30. Poon, J., Dryja, T.: The bitcoin lightning network: Scalable off-chain instant payments. `https://lightning.network/lightning-network-paper.pdf` (2016)
31. Roose, S.: The Ark case for CTV. Delving Bitcoin, `https://delvingbitcoin.org/t/the-ark-case-for-ctv/1528` (2025)
32. Rubin, J.: BIP-119: CHECKTEMPLATEVERIFY. `https://github.com/bitcoin/bips/blob/master/bip-0119.mediawiki` (2020)
33. Rubin, J.: BIP-449: OP_TWEAKADD, x-only key tweak addition. `https://github.com/bitcoin/bips/blob/master/bip-0449.md` (2026)
34. Russell, R.: OP_TX. Draft BIP on a personal fork, `https://github.com/rustyrussell/bips/blob/guilt/varops/bip-unknown-optx.mediawiki` (2024)
35. Russell, R., Moik, J.: BIP-440: Varops budget for script runtime constraint. `https://github.com/bitcoin/bips/blob/master/bip-0440.mediawiki` (2025)
36. Russell, R., Moik, J.: BIP-441: Restoration of disabled script (tapleaf 0xC2). `https://github.com/bitcoin/bips/blob/master/bip-0441.mediawiki` (2025)
37. Sanders, G.: LN-Symmetry (eltoo) transaction formats, draft. `https://github.com/instagibbs/bolts/blob/eltoo_draft/XX-eltoo-transactions.md` (2023)
38. Towns, A.: TAPLEAF_UPDATE_VERIFY covenant opcode. bitcoin-dev mailing list, 9 Sep 2021, `https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2021-September/019419.html` (2021)
39. Wuille, P., Nick, J., Ruffing, T.: BIP-340: Schnorr signatures for secp256k1. `https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki` (2020)
40. Wuille, P., Nick, J., Towns, A.: BIP-341: Taproot: SegWit version 1 spending rules. `https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki` (2020)
41. Wuille, P., Nick, J., Towns, A.: BIP-342: Validation of taproot scripts. `https://github.com/bitcoin/bips/blob/master/bip-0342.mediawiki` (2020)

Table 3. Deployment assumptions (spec 4.1, `spec/layertree.md`). A proposal *defines* its opcode for one tapleaf version; a tier may *assume* it in another.

	assumption	tiers
[A-1]	`OP_TX` [34] (a draft on a personal fork, defined for `0xC2` with BIP-440 costs) pushes amounts as 8-byte little-endian strings, which FULL compares with BIP-441 numerics	FULL, MID, CCV
[A-2]	CSFS (BIP-348, defined for `0xC0`) available in `0xC2` at the cost of one signature check	FULL, CCV
[A-3]	the EC-ops draft [25] available in `0xC2` at its stated costs	FULL
[A-4]	opcode numbers: `OP_TX` and `OP_EC_P0INT_NEGATE` both claim `0xbd`, BIP-443 and `OP_EC_P0INT_ADD` both `0xbb`; the simulator parks `OP_TX`, `OP_TWEAKADD` and CCV at `0xbf`, `0xc0`, `0xc1`, so the golden scripts are byte-exact under that numbering only (sizes are unaffected)	all
[A-5]	`OP_TX` in `0xC0` with `CScriptNum` outputs, which no proposal defines	MID, CCV
[A-6]	BIP-443 (defined for `0xC0`) also in `0xC2`; its sigop cost, open upstream, charged as one signature check; amount rules as of `bitcoin/bips` commit `24e96e87`. Conservation rests on the tier's `OP_TX` equalities, so a change to BIP-443 can only make honest spends fail	CCV
[MIN]	`OP_CHECKSIG` under key G with $s = e + 1$ accepts only if the script-built message is the real one, barring a collision in the TapSighash or challenge hash	MIN
[C]	contention: κ and β of §6 (policy and fees, not consensus)	all

42. Zhao, G.: BIP-431: Topology restrictions for pinning. `https://github.com/bitcoin/bips/blob/master/bip-0431.mediawiki` (2024)
43. ZmnSCPxj: OP_EVICT: An alternative to OP_TAPLEAFUPDATEVERIFY. bitcoin-dev mailing list, 18 Feb 2022, `https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2022-February/019926.html` (2022)
44. ZmnSCPxj: SuperScalar: Laddered timeout-tree-structured Decker-Wattenhofer factories. Delving Bitcoin, `https://delvingbitcoin.org/t/superscalar-laddered-timeout-tree-structured-decker-wattenhofer-factories/1143` (2024)

A Assumptions and Measured Sizes

Table 3 lists the deployment assumptions of §7, and Table 4 the script, witness and transaction sizes behind Figure 3 and Table 2.

B Costs Against the Alternatives

Joining, updating and leaving a shared output each cost more bytes than the same operation on a bare UTXO. What the sharing buys is a different placement of the cost. Entry is close to free in bulk, value moves off-chain for nothing, a bulk exit costs about a bare spend, and a solo exit costs about nine times one but is a single transaction with no timelock and no counterparty. This appendix walks through each of those claims with the measured figures, then says where LayerTree loses and how the numbers are made (§B.9).

B.1 The Unit of Cost

Every LayerTree operation is one transaction that spends the tree output and recreates it. The witness carries a leaf and its Merkle path, and the script folds

Table 4. Measured EXIT_1 cost (FULL unless noted) on the model of §10. Witness weight is script, witness data and a 129-byte control block for the eight-leaf reference script set (each doubling of the set adds 8 vB); the transaction adds the 946 WU skeleton. The sparse witness is $244 + 41k$ bytes for k non-empty siblings: $k = 2$ on the measured six-leaf tree (326 B), and $k \approx \log_2 N$, so the rows give the estimate at $N = 2^{16}$ ($k = 16$). Last column: unamortised radix-2 transaction tree of 2^D leaves, $150D + 111$ vB; sparse D is address width.

profile	D	script (B)	witness data (B)	witness (WU)	tx (vB)	radix-2 tx tree (vB)
dense	8	1,397	574	2,100	762	1,311
dense	12	1,749	738	2,616	891	1,911
dense	16	2,101	902	3,132	1,020	2,511
dense	20	2,453	1,066	3,648	1,149	3,111
sparse	32	5,147	≈ 900	≈ 6,176	≈ 1,781	n/a
sparse	64	9,499	≈ 900	≈ 10,528	≈ 2,869	n/a
MID dense	16	3,367	978	4,474	1,355	
MIN dense	16	3,640	1,175	4,944	1,473	
CCV dense	16	1,716	838	2,683	908	

the path twice, once to check the old root and once to compute the new one. Each level costs 88 bytes of script and 41 of witness in FULL, 129 weight units or about 32 vB. The depth D is fixed when the tree is created, so an exit from a given tree is a constant size whatever the population: 1,020 vB at $D = 16$, which holds 65,536 slots.

Figure 4 splits the exit into its parts, and Table 5 lists every variant against a bare spend. MID and MIN pay more for the same work because they lack the restored arithmetic and point opcodes; CCV saves about a tenth by letting `OP_CHECKCONTRACTVERIFY` bind the state. The hypothetical native Merkle-sum opcode of spec 13 item 12 is a measurement, never a tier: on the simulator's six-leaf tree its exit is 388 vB, with 252 bytes of script at any depth, but the witness still carries one 40-byte sibling per occupied level, so a full depth-16 tree costs about 520 vB. That is the floor for any design that publishes a proof and checks it in script. The transaction-tree row is the model of §10: D transactions of about 150 vB to unroll the branch, plus a 111 vB claim. It flatters the tree, since it omits the relative timelock at the leaf and the fee each of the 16 transactions pays separately; Table 6 gives the same model at other round sizes and radix 4.

B.2 Exits in Bulk

A single exit is the expensive case. The cost per leaf falls as more leaves leave in one transaction, and how far it falls depends on who has to sign (Figure 5, Table 7).

Owners who coordinate leave together with EXIT_K. Each leaf still carries its own path, so EXIT_2 is 1,684 vB: 842 vB a leaf, and 665 vB for the second exit alone. *The operator* sweeps an aligned cohort of expired leases with $\mathsf{EVICT_SUBTREE}_h$, which publishes the cohort's leaves once and shares the path above it: 203 vB a leaf for 13 leaves at $h = 4$, 158 vB for 51 at $h = 6$, and about 147 vB for the 205 leaves of a height-8 cohort at $D = 20$. The sweep needs no member, but it only

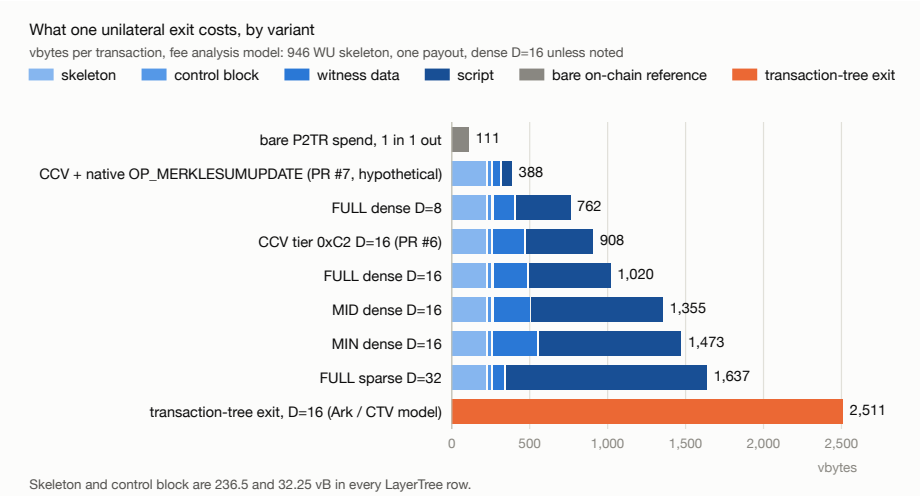


Fig. 4. One unilateral exit by variant. The skeleton (236.5 vB) and control block (32.25 vB) are the same in every LayerTree row; the script dominates because tapscript has no loops and the levels are unrolled. The sparse and native-opcode rows are measured on the simulator's six-leaf tree.

Table 5. One exit, dense $D = 16$ unless noted. Model: the transaction model of §B.9. Serialised: the real transaction the test suites build, with the witness length prefixes the model leaves out (`spec/scripts/weights.md`).

variant	model (vB)	serialised (vB)
bare P2TR spend, one input and one output	111	
one P2TR output inside a batch transaction	43	
FULL EXIT_1	1,020	1,030
FULL, fee taken from the payout	921	
FULL, $D = 8$ and $D = 20$	762 and 1,149	
FULL tower exit EXIT_1^t	1,046	
FULL sparse $D = 32$, six-leaf tree	1,637	1,648
FULL sparse $D = 32$, 2^{16} members (estimate)	$\approx$ 1,780	
MID	1,355	1,398
MIN	1,473	1,518
CCV, tapleaf `0xC2`	908	918
native Merkle-sum opcode (hypothetical), six-leaf tree	388	
native Merkle-sum opcode (hypothetical), full $D = 16$ tree	$\approx$ 520	
radix-2 transaction-tree exit (model)	2,511	

Table 6. The transaction-tree model across round sizes N. A level is one key-path P2TR input and r P2TR outputs, $68 + 43r$ vB (154 at radix 2, used as 150; 240 at radix 4); the claim is a plain P2TR spend, 111 vB; anchors and their fee children are omitted, which favours the tree. Mass exit: the tree's interior shared by all members, against LayerTree's $\mathsf{REWRITE}_6$ or sweep of a 64-slot cohort. LayerTree is measured at $D = \log_2 N$.

	LayerTree, dense	radix-2 tree	radix-4 tree
isolated exit, $N = 2^8$	762	1,311	1,071
isolated exit, $N = 2^{12}$	891	1,911	1,551
isolated exit, $N = 2^{16}$	1,020	2,511	2,031
mass exit, per member	158 to 162	≈ 261	≈ 191

Table 7. Batch shapes at dense $D = 16$ on the transaction model: vB per leaf that leaves or changes, or per member that joins. Self-funded joins add each member's P2TR key-path funding input (58 vB) and the signing round of §4.1. The $\mathsf{REWRITE}_h$ rows are FULL's figures.

removal	leaves	tx	per leaf	addition or change	leaves	tx	per leaf
EXIT_1	1	1,020	1,020	JOIN_0	1	937	937
EXIT_2	2	1,684	842	JOIN_2	4	872	218
$\mathsf{EVICT_SUBTREE}_4$	13	2,638	203	JOIN_8 (operator-funded)	256	679	2.7
$\mathsf{EVICT_SUBTREE}_6$	51	8,057	158	sparse JOIN_0, $D = 32$	1	1,601	1,601
$\mathsf{REWRITE}_2$, all leave	4	1,455	364	$\mathsf{REWRITE}_6$, keys rotated	64	9,496	148
$\mathsf{REWRITE}_6$, all leave	64	10,392	162	UPDATE, one leaf	1	1,030	1,030

works on leaves whose evictor is the operator and whose lease has run out. A leaf with the NUMS evictor, or with a lease far in the future, blocks the sweep of its whole cohort (spec 7.7), so the operator's cheap removal is something members opt into at join time by accepting a lease.

Owners who want the sweep's price without a lease use the cohort rewrite $\mathsf{REWRITE}_h$ (§4) with every position leaving. It is the same post-order fold, authorised by one plain signature per leaf and the operator's, with no nonce round. At $D = 16$ four owners leaving a height-2 cohort pay 364 vB each, sixteen at $h = 4$ pay 200 vB, and sixty-four at $h = 6$ pay 162 vB, within four vbytes of the operator's sweep. The same script settles a cohort in place: rotating every key of a height-6 cohort is 148 vB per changed leaf against 1,030 vB for an UPDATE each, and a position that does not change costs about 92 vB and needs no signature, so an absent member is carried through rather than excluded.

Batching changes what the chain limit counts. Every transition spends the same output, so unconfirmed transitions form a chain, and relay policy caps it: 25 transactions under the ancestor and descendant limits of Bitcoin Core before 31.0 [3], at most 64 under cluster mempool [9] (κ of §6). The cap counts transactions, not leaves, so a 51-leaf sweep is one entry. About 25 exits a block is the worst case, strangers leaving one at a time; a deployment that expects more shards into several trees, each sized by exit throughput rather than by depth. Each batch shape is its own tapleaf, and each doubling of the taptree adds 8 vB

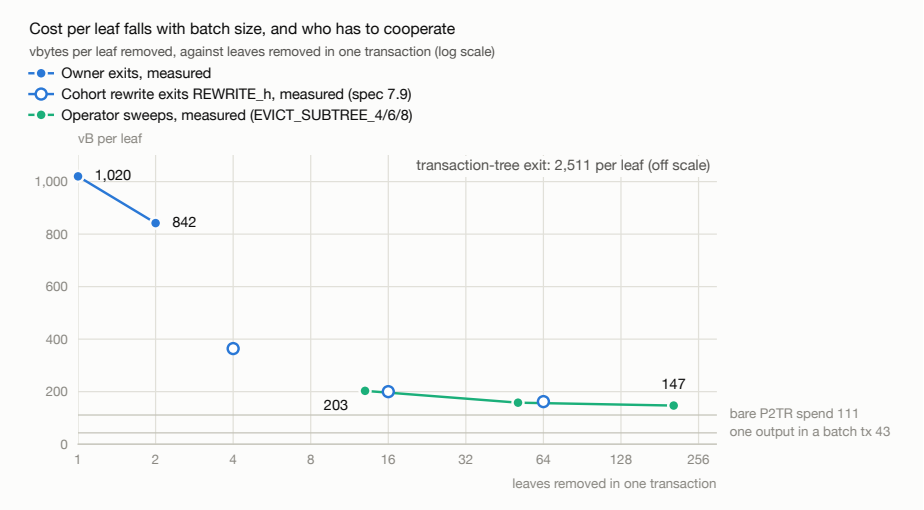


Fig. 5. Cost per leaf against leaves removed in one transaction (log scale): owner exits (EXIT_1, EXIT_2), owners leaving by $\mathsf{REWRITE}_h$ (364, 200 and 162 vB at 4, 16 and 64 positions), and operator sweeps ($\mathsf{EVICT_SUBTREE}_{4,6,8}$: 13, 51 and 205 leaves).

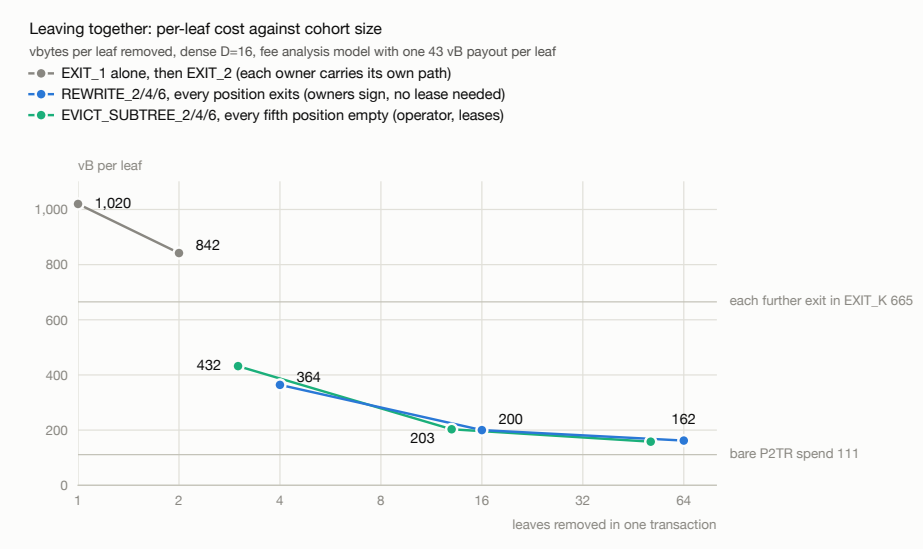


Fig. 6. The same three ways out at $D = 16$, per leaf removed against the number removed in one transaction: owners alone or in pairs, owners leaving together by $\mathsf{REWRITE}_h$, and the operator's sweep. The rewrite starts below the EXIT_K floor and meets the sweep near $h = 6$ (162 against 158 vB per leaf).

to every spend, so a tree that carries the sweeps and rewrites (16 to 32 tapleaves) pays 8 to 16 vB more on every exit.

B.3 Entry

JOIN_h grafts a prebuilt subtree of height h into an empty slot and walks only the $D - h$ levels above it, so a bigger cohort is a cheaper transaction (Figure 7). At $D = 16$ with no payout output, as spec 11 prices a join, one member costs 937 vB, a four-member graft 872 vB (218 vB a member), a 256-member graft 679 vB (2.7 vB) and a 1,024-member graft about 614 vB (an estimate, 0.6 vB), against 43 vB for one output in an on-chain batch and about 154 vB for a channel opened in its own transaction.

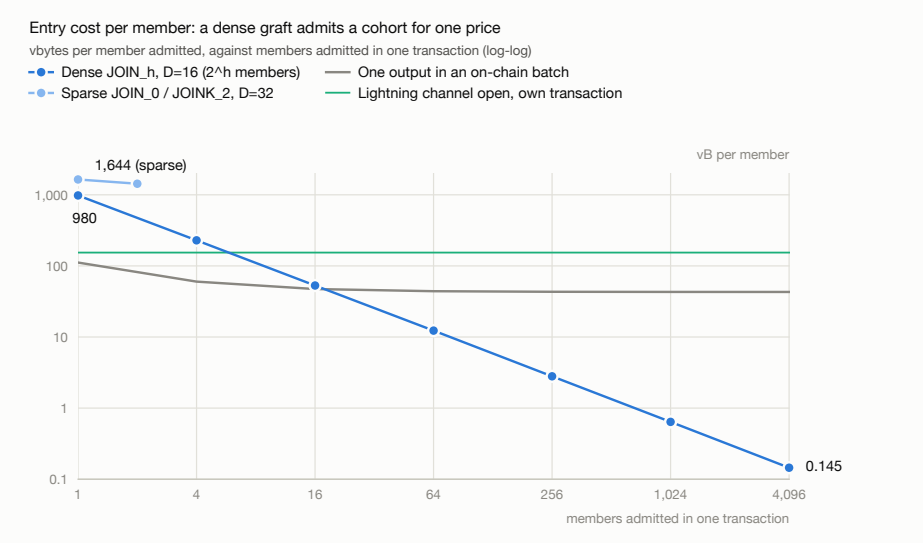


Fig. 7. Entry cost per member against members admitted in one transaction (log-log). The figure prices each join with one payout output, 43 vB above the no-payout pricing of spec 11 that the text and Table 7 use: its 980 and 1,644 are 937 and 1,601 there. The dense points above 4 members are extrapolated from the measured JOIN_2 at 129 weight units a level.

Who funds the value grafted in decides who has to be online. When the operator funds it from the transaction's fee input, the graft needs the operator's signature alone, and the whole cohort costs nothing beyond the figure; members pay for their balances off-chain after the graft confirms, or beforehand with a hash-locked voucher (§5). When each member funds its own leaf, each adds a key-path input of about 58 vB, the saving against an on-chain batch shrinks to the output side, and the cohort becomes a signing round among the funders:

every signature commits to the new root, so a joiner who drops out invalidates the others and the operator collects again (§4.1). Either way, a member checks its own path inside the subtree before funding, because a grafted node with no real tree behind it freezes its value. The sparse profile cannot graft at all: a sparse join is one leaf for 1,601 vB.

B.4 Moving Value

On-chain transitions are the wrong path for payments. An UPDATE at $D = 16$ is 1,030 vB and a voucher redemption with CLAIM 1,617 vB, priced as spec 11 prices them, without a payout output (1,073 and 1,660 vB on the skeleton that carries one), against 111 vB for a bare spend. A member who moved value by rewriting its leaf would pay about nine times what separate UTXOs pay per payment (Table 8).

Value moves off-chain instead, and the chain carries only membership changes and the exercise of exit rights. Payments between channel leaves are ordinary HTLC forwards; their funding outputs do not exist until a leaf exits (§9). A voucher is one signed message from payer to payee, credit until redeemed, and an issuer usually settles it off-chain rather than the holder redeeming it (§5). UPDATE is maintenance: resizing a balance, renewing a lease, rotating a key. When a cohort's balances do have to settle on-chain, $\mathsf{REWRITE}_h$ does it for about 148 vB per changed leaf with only the changed leaves' owners signing, which is the settlement an Ark round performs without the round.

Table 8. 1,000 members who each make 1,000 payments, entry and exit included where a design has them. The Lightning row opens and closes each channel cooperatively in its own transaction; the Ark row boards and leaves as outputs of round transactions, whose shared overhead is left out; the LayerTree rows enter by one 1,024-slot graft.

how value moves	on-chain bytes
bare UTXOs, one spend per payment	111 MvB
LayerTree, one on-chain UPDATE per payment	1,030 MvB
Lightning channels, payments off-chain	308 kvB
Ark, payments off-chain, plus a round transaction per refresh	86 kvB
LayerTree, payments off-chain, members leave by cohort exit or sweep	about 160 kvB
LayerTree, payments off-chain, members never exit (balances sold by voucher)	under 1 kvB

B.5 The Lightning Baseline and Lifetime per Member

The Lightning figures are a simple taproot channel [27] priced on the same model as the LayerTree rows, fee-bumping children included (Table 9). The non-taproot anchor commitment of BOLT 3 [19] is 281 vB in place of the 240 used here.

Against this baseline a member who leaves a dense $D = 16$ tree alone pays 1,020 vB, about twice a channel's force close and 1.5 times its 684-byte lifetime. A channel leaf inside the tree costs its exit plus an update, a settlement and

Table 9. A simple taproot channel on the transaction model.

transaction	vB
funding: one P2TR key-path input, the funding output and change	154
cooperative close: the MuSig2 key-path spend of the funding output	154
commitment: a MuSig2 key-path input, two balance outputs and two anchors	240
anchor child that pays the commitment's fee	152
delayed sweep of the broadcaster's output after the CSV	138
force close: commitment, anchor child and sweep	530
lifetime, open alone and force close	684
lifetime, open alone and cooperative close	308
lifetime, open as one output of a batch and cooperative close	197
lifetime, open as one output of a batch and force close	573

Table 10. Lifetime on-chain bytes per member, by how it leaves, and the unilateral exit's latency (a tower watches for stale states; an Ark member must refresh before expiry). LayerTree rows are dense at $D = 16$ unless marked sparse. Entry is one 43 vB output in a shared transaction; for LayerTree it is a share of a 256-slot operator-funded graft, or that plus the member's 58 vB input when self-funded. LayerTree's batch exit is the 51-leaf sweep, which requires a lease. The sparse row enters by one sparse JOIN_0 (1,601 vB, no payout output) and exits at the 2^{16}-member estimate (1,781 vB), not the six-leaf measurement. Lightning is a simple taproot channel: cooperative close 154 vB; force close a 240 vB commitment, its 152 vB anchor child and a 138 vB sweep. Ark's unilateral exit is the radix-2 unroll of its round.

	vB per member, leaving			unilateral exit		
	never alone	in a batch	alone	txs	blocks	online
LayerTree, operator-funded	3	162	1,023	1	1	nobody
LayerTree, self-funded	61	220	1,081	1	1	nobody
LayerTree sparse, $D = 32$	1,601	3,209	3,382	1	1	nobody
Lightning channel	197	197	573	3	≈ 145	tower
Ark, round of 256	86	86	1,354	9	8+ timelock	member
Ark, round of 65,536	86	86	2,554	17	16+ timelock	member
bare UTXO	154	154	154	1	1	nobody

their two anchor children, about 1,681 vB in all, against 684 vB for a channel with its own funding output, because materialising the funding output carries the Merkle-sum proof. The fair comparison adds entry and exit for one member and asks how the member leaves (Figure 8, Table 10).

Lifetime and latency. Table 10 adds entry and exit. The dense tree wins outright for a member who never exits alone or who is swept, and loses to Lightning and to a bare UTXO for a member who exits alone, though it beats an Ark unroll there; the sparse profile is the most expensive design in every column. Members who hold a lease and let the operator sweep them, or who sell their balance by voucher and never exit, pay almost nothing; members who insist on leaving alone pay for the proof.

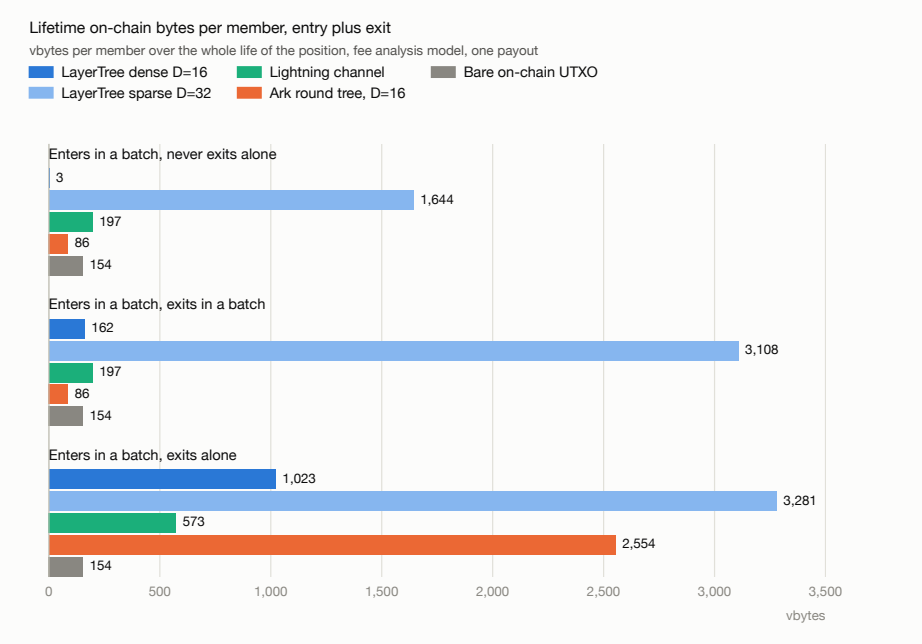


Fig. 8. Lifetime on-chain bytes per member, entry plus exit. The dense, Lightning, Ark and bare rows are those of Table 10. The sparse bars price the join with a payout output and the exit on the six-leaf tree (1,644, 3,108 and 3,281); the table prices the join without one and the exit at 2^{16} members (1,601, 3,209 and 3,382).

B.6 Time and Liveness

Bytes are half of an exit's cost. The other half is how long it takes and who has to be online (Table 11). A LayerTree exit has no pre-signed transaction to lose, no counterparty to wait for, no timelock and no expiry: the covenant checks a proof and pays. A plain exit's signature commits to the tree outpoint, which every operation changes, so the owner has to be online to build it and to rebuild it after a competing spend. The tower exit EXIT^t_K (§4) lifts that: the owner signs the leaf and the tree's identifier once, in advance, and whoever holds the signature builds and rebuilds the exit against whatever root is current. A channel leaf gives up some of the rest in exchange for free payments.

Table 11. The unilateral exit's latency and liveness. Ark is a round of 2^{16} members; the channel leaf and the Lightning channel close by force.

	LayerTree EXIT_K	LayerTree EXIT^t_K	LayerTree channel leaf	Lightning	Ark
transactions	1	1	3, plus fee children	3: commitment, anchor child, sweep	16, plus the claim
blocks until spendable	1	1	about 146	about 145	16 plus the leaf timelock
timelock	none	none	CSV on the settlement	CSV on the commitment	relative lock at the leaf
who must be online	the owner, to build, sign and rebuild if outbid	nobody: a tower holds a one-time authorisation	the closer; a tower can finish	a tower, for stale states	the member, before the round expires
what expires	nothing; leases are optional	nothing; a changed leaf voids the authorisation	nothing	nothing	the round: refresh or forfeit

The exit is one column of a wider question: who has to be awake for each thing a member does. Table 12 answers it for the designs of §11, and Table 13 sets their structure side by side. Across LayerTree's row the only party ever needed for *another* member's operation is the operator, and only to add members, co-sign an UPDATE or settle a cohort; no member is needed for another's exit, join, payment or eviction. Pre-signed designs need the whole set at creation because the interior is signatures, and the round-based ones make every member come back before an expiry. Covenant trees remove the signatures but keep the $O(\log N)$ exit. The pools match the one-transaction exit, each with a purpose-built opcode and with every change to the pool signed by every participant.

B.7 Dense or Sparse

The sparse profile is a key-value store: a leaf's position is a hash of its owner key and nonce, so nobody allocates slots, the root does not depend on insertion order, and absence is an inclusion proof of an empty node. It is not the cheaper

Table 12. Who has to be awake, by operation. "All" is every member of the shared output; a round is a signing session that restarts if one signer drops.

	join	pay off-chain	settle on-chain	exit	evict	expiry
LayerTree	operator alone, or a round among self-funders	one message (voucher), or two parties (channel)	UPDATE: one exchange; $\mathsf{REWRITE}_h$: the changed owners	alone, 1 tx	operator alone, after the lease; member paid	none
Lightning [30]	two parties	two parties	two parties	alone	n/a	none
Factory [6]	all	two parties, in a channel	all, every state	$O(\log N)$ pre-signed txs	no	none
Ark, pre-signed [17]	the round, in a window	server co-signs, trusted until the next round	the next round, with everyone	$O(\log N)$ txs plus a timelock	no; the round expires	every round
Ark, covenant [31]	funders sign their inputs	as above	the next round	$O(\log N)$ txs	no; the round expires	every round
Timeout tree [18]	all at creation, or CTV	channels inside	a new tree	$O(\log N)$ txs	no; the tree expires	per tree
Payment pool [38,22,43]	all sign the new state	all sign	all sign	alone, 1 tx	`OP_EVICT`: the others sign	none
CTV or CCV tree [32,14]	operator builds the templates	varies	a new tree	$O(\log N)$ txs	no	per design

Table 13. Comparison with shared-output constructions. Pools: `OP_TAPLEAF_UPDATE_VERIFY` [38], CoinPool's `OP_MERKLESUB` [22], `OP_EVICT` [43]. GSR: the string opcodes BIP-441 restores; APO: `SIGHASH_ANYPREVOUT`. LayerTree's interactive signing: the covenant needs the operator only; a self-funded cohort's funders sign one transaction together.

	LayerTree	Pools	Ark (deployed)	Ark (covenant)	Timeout tree	Factory
Unilateral exit	1 tx, $O(D)$ bytes	1 tx	$O(\log N)$ txs	$O(\log N)$ txs	$O(\log N)$ txs	$O(\log N)$ txs
Evict one member	1 tx after its lease	1 tx (`EVICT`)	round expires	round expires	tree expires	no
Member position expiry	lease, renewed by the operator; eviction pays the member	none	per round; operator reclaims	per round	per tree	none
Interactive signing	operator only; a self-funded cohort once	all, to update	round participants	funders, per round	none or all	all
Shared output expiry	none	none	per round	per round	per tree	none
Batch add	1 tx, any subtree	re-sign	per round	per round	per tree	re-sign all
Opcodes	FULL: GSR, TX, CSFS, EC; MID: CAT, TX, CSFS, TWEAKADD; MIN: CAT, TWEAKADD; CCV: CCV, TX	a pool opcode (and APO for CoinPool)	none	CTV or CCV	CTV	none

profile on-chain (Figure 9). A sparse proof elides empty siblings to one byte, so its witness is small, 326 bytes at $D = 32$ on the six-leaf tree against 902 for dense at $D = 16$. But the script dominates. Random positions need about twice the depth of allocated ones to keep collisions rare, and a sparse level is 135 bytes of script against 88, because it derives the direction bit from the position and computes the empty node. The exit is 1,637 vB on six leaves and about 1,780 vB at 2^{16} members, 1.75 times dense, and the profile gives up grafts and sweeps. So the sparse profile buys properties, not bytes. Raising the radix does not help either: sibling bytes per exit grow from $41D$ at radix 2 to about $61.5D$ at radix 4, and a measured radix-4 level brings the dense exit to 1,042 vB and radix 8 to 1,257 vB, against 1,020 vB at radix 2. With a native opcode the per-level cost would be the sibling alone, and radix 2 would be optimal outright.

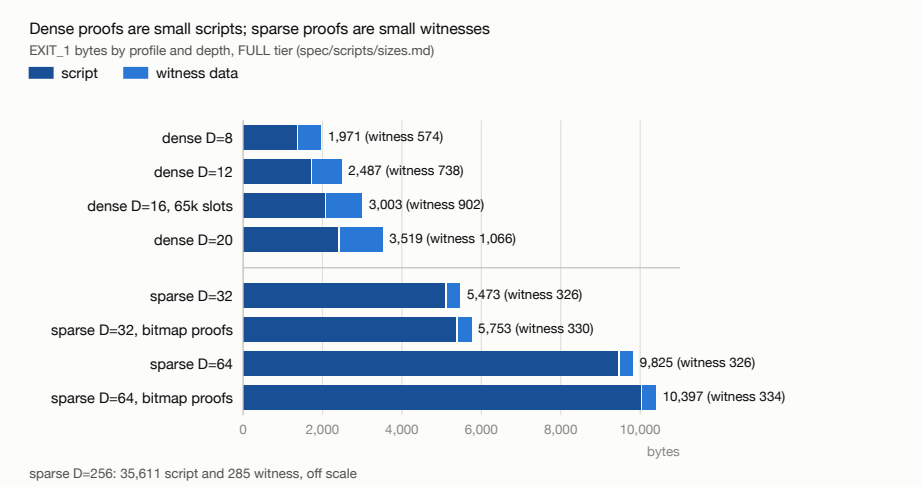


Fig. 9. EXIT_1 bytes by profile and depth, FULL tier: dense proofs are small scripts, sparse proofs small witnesses. Sparse rows are measured on the six-leaf tree.

B.8 Where LayerTree Loses

A member who never leaves alone pays almost nothing: about 3 vB for a share of a graft, and about 160 vB in all if it leaves with its cohort. A member who does leave alone pays 1,020 vB, about nine times a bare spend and about twice a Lightning force close; the proof is the price of the shared output. An on-chain update is 1,030 vB, so payments must be off-chain or batched. Everything serialises through one output, at about 25 uncoordinated exits a block in the worst case, which bounds how many time-sensitive leaves one tree should host; fee pinning on that output is outside the model. The sparse profile costs 1.7 to 2.8 times the dense one and cannot graft or sweep. The MID tier caps a tree at 2^{31} satoshis. And

the opcodes do not exist: every tier needs proposals that are not active, three of them in a tapleaf version their proposals do not define (Table 3).

B.9 How the Numbers Are Made

Every figure is a whole transaction on one model, the same in spec 11, `spec/fee-analysis.md`, the evaluation of §10 and here: the measured script and witness and a 129-byte control block (the eight-leaf reference taptree), plus a 946 WU skeleton (the tree input, a P2WPKH fee input, the tree output, one P2TR payout and a P2WPKH change output) and 172 WU per further payout, with $\text{vB} = \lceil \text{WU}/4 \rceil$ once per transaction. A transition without a payout (JOIN, UPDATE, CLAIM, SPLIT, a rewrite that only updates) is priced on 774 WU, and the fee-from-payout exit on 550 WU. The model leaves out the witness item count and the per-item length prefixes; `tools/measure_weight.py` serialises the transactions the suites build and measures them (`spec/scripts/weights.md`): 10.5 vB on a FULL EXIT_1 at $D = 16$, 10 in CCV, and 42.5 and 44.75 in MID and MIN, whose witnesses carry each sibling as several small elements. The sizes come from `tools/gen_listings.py` (`spec/scripts/sizes.md`), the fee tables from `tools/fee_analysis.py`, and the figures of this appendix from `tools/gen_cost_figures.py`; the baselines are models, not measurements. The suites behind the sizes run 1,154 FULL, 466 MID, 55 MIN, 1,344 CCV and 220 channel-leaf checks, and 61 for the hypothetical opcode, which stays out of every tier.